Gradient Descent Algorithm

Chapter 4: **Training Models**
Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, Gurelien Geron, 3e, O'Reilly Media, 2022.

Optimization

- Optimization is the problem of finding inputs to an objective function that produce the best outcome, either a maximum or a minimum.
- Optimization space can discrete where the inputs are chosen from a finite or countable set, or continuous where the inputs take real values.

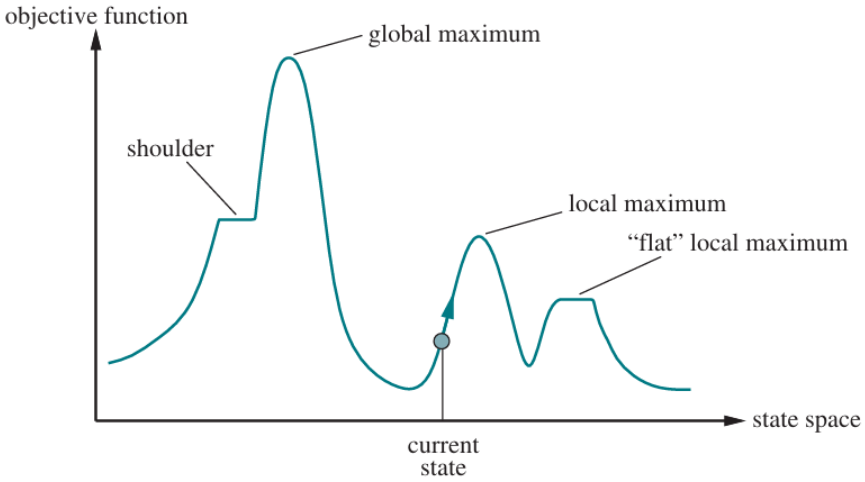
Objective Function

- The objective function is the function to be optimized (minimized or maximized).
- We can visualize the search space as a landscape:
  - x-axis: states (possible solutions).
  - y-axis: objective function value (elevation).
- Each state corresponds to an elevation; could be a heuristic value or some other quality measure like cost or maximum likelihood or utility.

State-Space Objective Function Terminology

- **Local Maximum:** A state better than all its immediate neighbors, but not necessarily the best overall.
- **Global Maximum:** The highest possible state in the entire search space.
- **Local Minimum:** A state worse than all its immediate neighbors, but not necessarily the worst overall.
- **Global Minimum:** The lowest possible state in the entire search space.
- **Plateau:** A flat region of the landscape where many neighboring states have the same value.
- **Shoulder:** A plateau with an uphill path leading out of it.

- If the goal is maximization, find the highest peak (global maximum). Algorithms like hill climbing or simulated annealing attempt this.
- If the goal is minimization, find the lowest valley (global minimum). Gradient descent is commonly used here.
- Local optima may trap simple algorithms, which is why advanced methods (simulated annealing, evolutionary algorithms) are valuable.

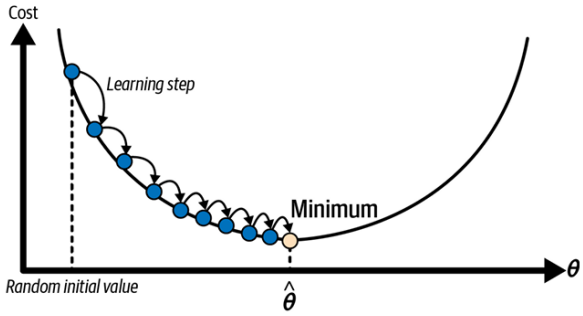


The Gradient Descent Algorithm

- Gradient descent is an efficient first order optimization algorithm that attempts to minimize the cost function by finding a local or global minima of the function.
- In general it can be used to minimize any function.
- Concretely, it starts by initializing the training parameters randomly, and then improving them gradually, taking one baby step at a time, each step attempting to decrease the cost function, until the algorithm converges to a minimum.

Training Gradient Descent with a Single Parameter

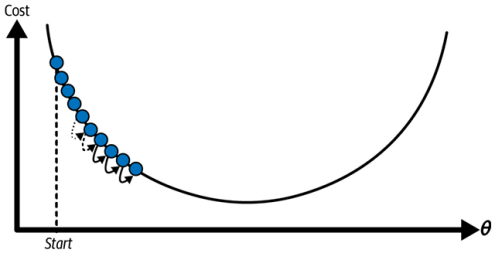
- Let θ be the parameter to be trained and $J(\theta)$ be the cost function.
- Initialize θ by choose a starting point in the parameter (weight) space (random initialization).
- Compute the gradient of the cost function at the current point.
- Move a small step in the opposite direction of the gradient (steepest descent).
- Repeat steps 2 and 3 until convergence.



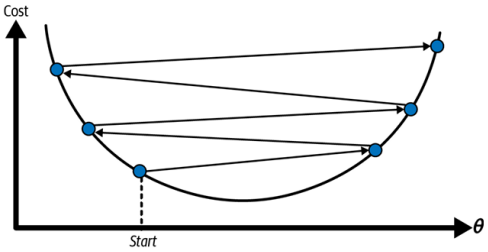
- The parameter update is given by:
  $\theta = \theta - \alpha \frac{d}{d\theta} J(\theta)$, where α is the learning rate, which controls the step size.
- A small learning rate results in smaller steps and slower convergence.
- A large learning rate results in larger steps and faster learning, but may overshoot the minimum and may fail to converge.
- As the algorithm approaches the minimum, the gradient becomes smaller, and steps automatically shrink.

- Gradient descent is said to converge when any one of the following conditions is met:
  - Maximum number of iterations is reached.
  - The change in parameters becomes very small.
    The norm of the gradient becomes smaller than a small threshold called tolerance or precision.

- Thus the gradient descent algorithm is:
  Repeat until converge:
  $$\theta = \theta - \alpha \frac{d}{d\theta} J(\theta)$$

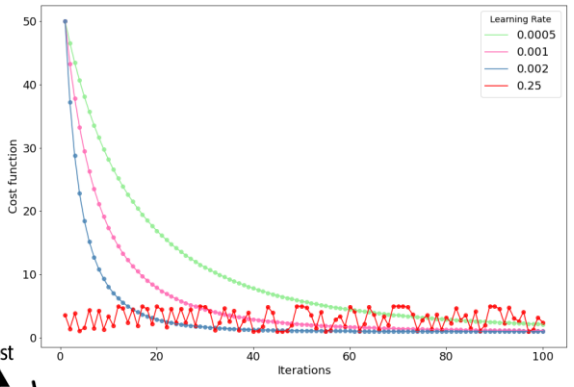


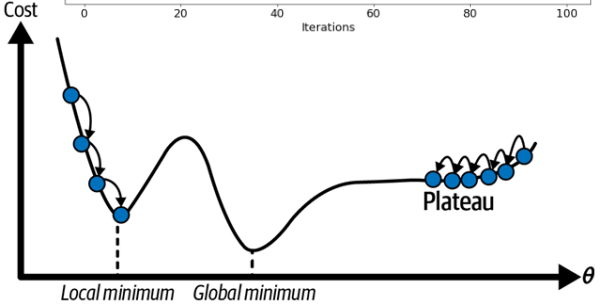
Very low learning rate



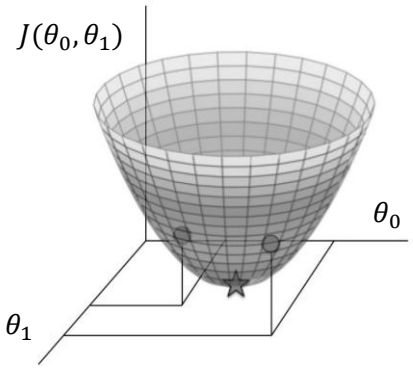
Very high learning rate

- A good way to make sure gradient descent runs properly is by plotting the cost function as the optimization runs.
- Put the number of iterations on the x-axis and the value of the cost-function on the y-axis.
- This helps visualization of the cost function after each iteration of gradient descent, and provides a way to easily spot how appropriate the learning rate is.
- Try different values for α it and plot them all together. If gradient descent is working properly, the cost function should decrease after every iteration.
- Near the minimum, gradient descent may take a long time to reach the exact minimum, but it gets very close.
- If the cost function has multiple minima, the final solution depends on the initial starting point, and the algorithm may converge to different local minima.
- If the cost function has plateau, then the algorithm will take a very long time to cross it; and if the algorithm stops too early, it will never reach the global minimum.





Training Gradient Descent Algorithm with Two Parameters



- Consider a model with two parameters, θ_0 and θ_1.
- The cost function $J(\theta_0, \theta_1)$ can be visualized as a three-dimensional surface, where the horizontal axes correspond to the parameters θ_0 and θ_1, and the vertical axis corresponds to the value of the cost function (error).
- Each point on the horizontal plane represents a particular setting of the model parameters, and the height of the surface at that point represents the incurred error.
- To apply gradient descent with multiple parameters, we compute the partial derivative of the cost function with respect to each parameter.
- A partial derivative measures how much the cost changes when one parameter is varied slightly while all others are held constant.
- Training a model can therefore be seen as a search in parameter space for the parameter values that minimize the cost function. As the number of parameters increases, the dimensionality of this space increases, making the search more challenging.

- Now the gradient descent algorithm is:

  Repeat until converge:

$$t_0 = \theta_0 - \alpha \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1)$$

$$t_1 = \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1)$$

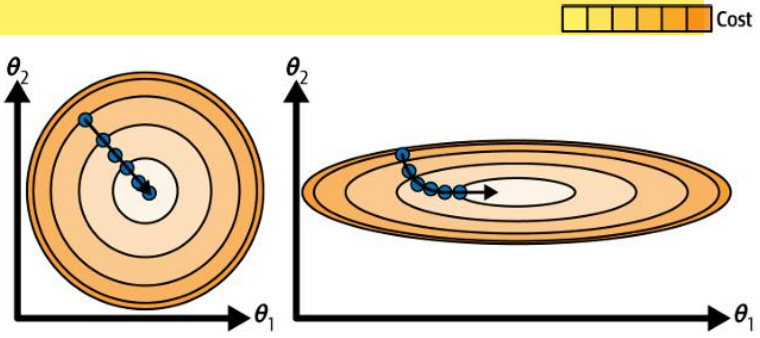
$$\theta_0 = t_0$$

$$\theta_1 = t_1$$

  - The parameters must be updated simultaneously at each iteration.
  - Updating one parameter before computing the update for the other would lead to an incorrect implementation of gradient descent.
  - Although convergence can be detected in multi-parameter settings, high-dimensional optimization may converge slowly, so a fixed iteration limit is commonly used in practice.

Contour Plots

- The three-dimensional cost surface can be conveniently visualized using contour (level) plots, where the minimum error lies at the center of elliptical contours.
- A contour line represents a curve along which the cost function has a constant value, connecting points of equal error.
- In a contour plot, the plane is two-dimensional and corresponds to the model parameters, while the direction of steepest descent is always perpendicular to the contour lines.
- This direction is represented by the gradient vector.
- Although the cost function typically has a bowl-shaped surface, it may become elongated when features have very different scales.
- When features are on similar scales, gradient descent moves directly toward the minimum and converges quickly.
- When feature scales differ significantly, gradient descent follows a zigzag path along a narrow valley, resulting in slow convergence.
- Therefore, feature scaling is important to ensure faster and more efficient convergence of gradient descent.



Generalization of Gradient Descent Algorithm

- In general, for a total of p parameters and cost function $J(\theta_0, \theta_1, \ldots\ldots, \theta_p)$ gradient descent algorithm is:

  Repeat until converge:

  $\theta_j = \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1, \ldots\ldots, \theta_p)$, where j represents the feature index number (parameter number).

- Most shallow learning algorithms, such as linear regression and logistic regression, use convex cost functions.
- A convex cost function has a single global minimum, which guarantees that gradient descent converges to the same solution regardless of parameter initialization (given an appropriate learning rate).
- These cost functions are typically continuous, smooth, and differentiable, meaning the gradient changes gradually and allowing gradient-based optimization methods to be applied effectively.
- A cost function with one parameter can be visualized as a 2D curve.
- A cost function with two parameters forms a 3D surface.
- For cost functions with more than two parameters, direct visualization becomes impractical.
- In contrast, non-convex cost functions contain multiple minima.
- In such cases, gradient descent may converge to different local or global minima, depending on the initial parameter values.